- **比较运算符的使用**
  - 比较运算符：
    - >：　左边是否大于右边
    - <：　左边是否小于右边
    - >=：　左边是否大于或等于右边
    - <=：　左边是否小于或等于右边
    - ==：　左右两边是否相等
    - ===：　左右两边是否类型和值都相等
    - != 不相等
    - !==：　左右两边是否不全等
  - 比较结果为boolean类型，即只会得到true或false

- **比较运算符的细节**
  - 字符串比较，是比较的字符对应的ASCII码
    - 从左往右依次比较
    - 如果第一位一样再比较第二位，以此类推
    - 比较的少，了解即可
  - NaN不等于任何值，包括它本身
    - 用isNaN函数判断是否为NaN
  - 尽量不要比较小数相等或全等的问题，因为小数有精度问题
  - 不同类型之间比较会发生隐式转换
    - 最终把数据隐式转换转成number类型再比较
    - 所以开发中，如果进行准确的比较我们更喜欢 === 或者 !==

ASCII 字符代码表 一

高四位		ASCII非打印控制字符											ASCII 打印字符												
		0000					0001						0010		0011		0100		0101		0110		0111		
		0					1						2		3		4		5		6		7		
低四位		+缺制	字符	ctrl	代码	字符解释	+缺制	字符	ctrl	代码	字符解释		+缺制	字符	+缺制	字符	+缺制	字符	+缺制	字符	+缺制	字符	+缺制	字符	ctrl
0000	0	0	BLANK NULL	^@	NUL	空	16	►	^P	DLE	数据链路转意		32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH	头标开始	17	◄	^Q	DC1	设备控制 1		33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☻	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2		34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX	正文结束	19	‼	^S	DC3	设备控制 3		35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4		36	$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ	查询	21	§	^U	NAK	反确认		37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲		38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	●	^G	BEL	震铃	23	↨	^W	ETB	传输块结束		39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◘	^H	BS	退格	24	↑	^X	CAN	取消		40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束		41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◙	^J	LF	换行/新行	26	→	^Z	SUB	替换		42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	垂直制表符	27	←	^[	ESC	转意		43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	换页/新页	28	∟	^\	FS	文件分隔符		44	,	60	<	76	L	92	\	108	l	124	|	
1101	D	13	♪	^M	CR	回车	29	↔	^]	GS	组分隔符		45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	♫	^N	SO	移出	30	▲	^6	RS	记录分隔符		46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符		47	/	63	?	79	O	95	_	111	o	127	△	Back space

注：表中的ASCII字符可以用:ALT + "小键盘上的数字键" 输入

- **逻辑运算符的使用**
  - 逻辑运算符：

符号	名称	日常读法	特点	口诀
&&	逻辑与	并且	符号两边都为true结果才为true	一假则假
||	逻辑或	或者	符号两边有一个true就为true	一真则真
!	逻辑非	取反	true变false false变true	真变假，假变真

- **逻辑运算符里的短路**
  - 短路：只存在于 && 和 || 中，当满足一定条件会让右边代码不执行

符号	短路条件
&&	左边为false就短路
||	左边为true就短路

  - 原因：通过左边能得到整个式子的结果，因此没必要再判断右边
  - 运算结果：无论 && 还是 || ，运算结果都是最后被执行的表达式值，一般用在变量赋值

```
console.log(false && 20)  // false
console.log(5 < 3 && 20) // flase
console.log(undefined && 20) // undefined
console.log(null && 20) // null
console.log(0 && 20) // 0
console.log(10 && 20) // 20
```

```
console.log(false || 20)  // 20
console.log(5 < 3 || 20) // 20
console.log(undefined || 20) // 20
console.log(null || 20) // 20
console.log(0 || 20) // 20
console.log(10 || 20) // 10
```

  - 经常用于函数传递参数的完整性

```
function getSum(x, y) {
    x = x || 0    // 如果左边没有传递参数过来就执行 0
    y = y || 0   // 如果传递参数过来，就执行参数
    return x + y
}
let sum = getSum(x, y)
console.log(sum)
```

- **if语句**
  - if语句有三种使用：单分支、双分支、多分支
  - 单分支使用语法：

```
if (条件) {
    满足条件要执行的代码
}
```

    - 括号内的条件为true时，进入大括号里执行代码
    - 小括号内的结果若不是布尔类型时，会发生隐式转换转为布尔类型

- 双分支if语法：

```
if (条件) {

    满足条件要执行的代码

} else {

    不满足条件执行的代码
}
```

- 多分支if语法：

```
if (条件1) {

    代码1
} else if (条件2) {

    代码2
} else if (条件3) {

    代码3
} else {

    代码n
}
```

  - 先判断条件1，若满足条件1就执行代码1，其他不执行
  - 若不满足则向下判断条件2，满足条件2执行代码2，其他不执行
  - 若依然不满足继续往下判断，依次类推
  - 若以上条件都不满足，执行else里的代码n
  - 注：可以写N个条件，但这里演示只写2个

- **switch语句**

```
switch (数据) {
    case 值1:
        代码1
        break
    case 值2:
        代码2
        break
    default:
        代码n
        break
}
```

- 找到跟小括号里数据全等的case值，并执行里面对应的代码
- 若没有全等的则执行default里的代码

- 例：数据若跟值2全等，则执行代码2
- break作用：立即结束所在的switch语句
- break可防止穿透，若不写会发生穿透效果
- 穿透效果：从一个case块进入到下一个case块

- **三元表达式**
  - 也是双分支，本质是if双分支的简写
  - 符号：？与：配合使用
  - 语法：

```
条件 ？满足条件执行的代码 ： 不满足条件执行的代码
```

  - 一般用来取值
  - 案例：判断两个数谁是更大数
  - 案例： 数字补0案例

- **术语解释**

术语	解释	举例
关键字	在JavaScript中有特殊意义的词汇	let、var、function、if、else、switch、case、break
保留字	在目前的JavaScript中没意义，但未来可能会具有特殊意义的词汇	int、short、long、char
标识（标识符）	变量名、函数名的另一种叫法	无
表达式	能产生值的代码，一般配合运算符出现	10 + 3、age >= 18
语句	一句代码也称之为一条语句，一般按用途还会分类：输出语句、声明语句、分支语句	无